

E-commerce Customer Segmentation (Clustering) – End-to-End Project

1. Problem Statement

The objective is to segment users of an e-commerce platform based on behavioral event data (views, carts, purchases) to uncover hidden patterns. This enables targeted marketing, improved conversion, and personalized user experience.

2. Business Objective & Thinking

Unlike regression, clustering does not predict a target variable but groups users based on similarity. The goal is to identify actionable segments such as:

- Window Shoppers → High browsing, low purchase
- High-Intent Buyers → Strong conversion signals
- Inactive Users → Low engagement

Each segment should directly translate into business actions like retargeting, premium recommendations, or re-engagement campaigns.

3. Data Understanding

The dataset is event-level, meaning multiple rows per user. Therefore, the first critical decision was to convert it into a user-level feature table. This is essential because clustering operates on entities (users), not events.

4. Feature Engineering (Core of Project)

We engineered behavioral features capturing different dimensions:

Feature	Insight	Business Meaning
total_view	Browsing intensity	User engagement level
total_cart	Purchase intent	Mid-funnel strength
total_purchase	Conversion	Revenue generation
avg_price	Spending level	Low vs high-value customers

unique_category	Exploration	Focused vs exploratory
unique_brand	Brand diversity	Brand loyalty vs switching

5. Derived Behavioral Features

Raw counts were insufficient. Hence ratio-based features were introduced:

- purchase_rate → conversion efficiency
- cart_rate → intent strength
- cart_to_purchase_ratio → cart abandonment behavior

These features were critical in separating buyers from browsers.

6. Key Observations (EDA)

- Majority users have zero purchases → highly imbalanced behavior
- Heavy right skew → few users dominate activity
- Low category diversity → users are mostly focused

Insight: dataset is dominated by low-engagement users, making clustering harder and requiring transformation.

7. Data Transformation Decisions

Log transformation was applied to handle skewness and compress extreme values without losing information.

Scaling was applied because KMeans relies on distance, and unscaled features would bias clustering.

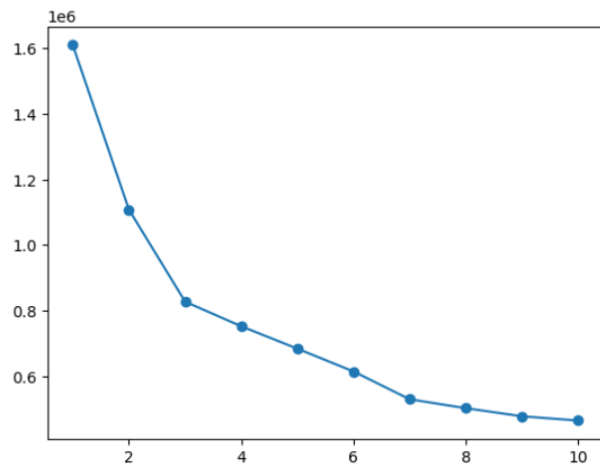
8. Model Selection & Trade-offs

KMeans, DBSCAN, and Agglomerative clustering were evaluated.

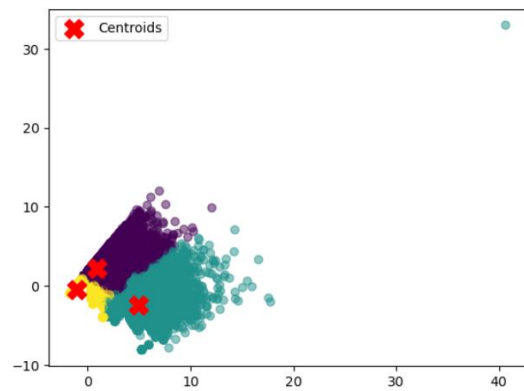
KMeans:

- Pros: scalable, interpretable
- Cons: assumes spherical clusters

```
[115]: from sklearn.cluster import KMeans
sse=[]
k_range=range(1,11)
for i in k_range:
    km=KMeans(n_clusters=i, random_state=42, n_init='auto')
    km.fit(X_scaled)
    sse.append(km.inertia_)
plt.plot(k_range, sse, marker='o')
plt.show()
```



```
)
plt.legend()
plt.show()
```



DBSCAN:

- Pros: handles noise
- Cons: poor performance on this dataset

Dbscan

```
[146]: from sklearn.cluster import DBSCAN  
db=DBSCAN(eps=0.5, min_samples=5)  
db_labels=db.fit_predict(X_scaled)
```

```
[147]: silhouette_score(X_scaled, db_labels, sample_size=10000, random_state=42)
```

```
[147]: np.float64(0.08993021891912042)
```

Final Decision: KMeans (Silhouette ≈ 0.39)

9. Cluster Interpretation

Cluster profiling revealed:

Cluster	Behavior	Action
0	High views, low purchase	Retargeting campaigns
1	High purchase rate	Premium offers
2	Low activity	Re-engagement

10. Production Pipeline

preprocessing.py → feature engineering

train.py → model training

```
C:\Users\shambhavic\OneDrive - Temenos\Data Science - e Project\Clustering\Ecommerce_behaviour\train.py
temp.py X train.py X predict.py X preprocessing.py X
4
5 @author: shambhavic
6
7
8 import pandas as pd
9 import pickle
10 import sqlite3
11 from sklearn.preprocessing import StandardScaler
12 from sklearn.cluster import KMeans
13 from preprocessing import preprocess_data
14
15 # load data
16 conn = sqlite3.connect("clustering.db")
17 df = pd.read_sql("SELECT * FROM ecommerce_behavior", conn)
18 conn.close()
19
20
21 # preprocess
22 user_df = preprocess_data(df)
23
24 # drop user_id if present
25 X = user_df.copy()
26 X = X.drop(columns=['user_id'], errors='ignore')
27
28 # scaling
29 scaler = StandardScaler()
30 X_scaled = scaler.fit_transform(X)
31
32 # model
33 km = KMeans(n_clusters=3, random_state=42, n_init='auto')
34 km.fit(X_scaled)
35
36 # save
37 import os
38 os.makedirs('models', exist_ok=True)
39 pickle.dump(km, open('models/kmeans_model.pkl', 'wb'))
40 pickle.dump(scaler, open('models/scaler.pkl', 'wb'))
41
42 print("Training complete!")
```

Code not profiled yet

Profile your code to explore which functions and methods took the longest to run and were called the most, and find out where to optimize it.

Help | Variable Explorer | Debugger | Plots | Files | Profiler

Console 1/A X train.py X

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy
A value is trying to be set on a copy of a slice from a DataFrame.
Try using `.loc[row_indexer,col_indexer] = value` instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy
A value is trying to be set on a copy of a slice from a DataFrame.
Try using `.loc[row_indexer,col_indexer] = value` instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy
A value is trying to be set on a copy of a slice from a DataFrame.
Try using `.loc[row_indexer,col_indexer] = value` instead

Training complete!

In [4]:

predict.py → inference

```
temp.py X train.py X predict.py X preprocessing.py X
1 # -*- coding: utf-8 -*-
2
3 Created on Sun Apr 26 01:42:11 2026
4
5 @author: shambhavic
6
7
8 import pickle
9 from preprocessing import preprocess_data
10
11 def predict_clusters(df):
12
13     # load artifacts
14     model = pickle.load(open('models/kmeans_model.pkl', 'rb'))
15     scaler = pickle.load(open('models/scaler.pkl', 'rb'))
16
17     # preprocess
18     user_df = preprocess_data(df)
19
20     # scale
21     X_scaled = scaler.transform(user_df.drop(columns=['user_id'], errors='ignore'))
22
23     # predict
24     clusters = model.predict(X_scaled)
25
26     user_df['cluster'] = clusters
27     user_df = user_df.reset_index()
28     return user_df[['user_id', 'cluster']]
29
30
31 if __name__ == "__main__":
32     import sqlite3
33     import pandas as pd
34
35     conn = sqlite3.connect("clustering.db")
36     df = pd.read_sql("SELECT * FROM ecommerce_behavior", conn)
37     conn.close()
38
39     result = predict_clusters(df)
40     print(result.head())
```

Code not profiled yet

Profile your code to explore which functions and methods took the longest to run and were called the most, and find out where to optimize it.

Help | Variable Explorer | Debugger | Plots | Files | Profiler

Console 1/A X train.py X predict.py X

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy
A value is trying to be set on a copy of a slice from a DataFrame.
Try using `.loc[row_indexer,col_indexer] = value` instead

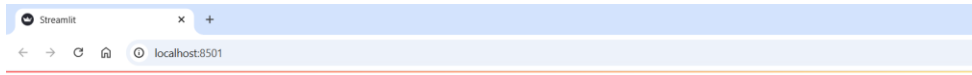
See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy
A value is trying to be set on a copy of a slice from a DataFrame.
Try using `.loc[row_indexer,col_indexer] = value` instead

user_id	cluster
0 104655840	1
1 128968633	0
2 176250879	1
3 178539771	1
4 288005640	1

In [5]:

Python Console | History

Streamlit app → UI layer



Customer Segmentation

Run Clustering

	user_id	cluster
0	104655840	1
1	128968633	0
2	176250879	1
3	178539771	1
4	208005640	1

11. Monitoring & Retraining Strategy

Clustering requires monitoring of:

- Feature drift
- Cluster distribution changes
- Silhouette score degradation

Retraining is triggered periodically or based on drift detection.

12. Conclusion

This project demonstrates not just clustering, but decision-making: feature design, model trade-offs, and production readiness. The focus was on extracting business value rather than just applying algorithms.